

Adaptive Multi-Resolution Grid-Based Clustering for Massive Data Streams

Alireza Zarei

Department of Mathematics and Computer Science
Sharif University of Technology
`zareai@sharif.ir`

Mohammad Saleh Bahrami

Department of Mathematics and Computer Science
Sharif University of Technology
`saleh.bahrami98@sharif.edu`

Abstract

The rapid growth of streaming data generated by sensors, Internet of Things devices, online platforms, and scientific applications has created a need for clustering algorithms that can process data online under strict memory constraints. Grid-based density clustering methods, such as D-Stream, are attractive for data stream clustering because they summarize incoming points using grid cells instead of storing the entire stream. However, fixed-resolution grid methods suffer from two major limitations: their memory consumption grows rapidly with dimensionality, and a single global cell size may be unable to capture clusters with different local densities or evolving structures.

In this paper, we propose an adaptive multi-resolution grid-based framework for clustering massive data streams. The method maintains a hierarchical grid structure in which each cell stores compact statistical summaries, including time-decayed density, center of mass, variance, and update time. Cells are dynamically refined when their current resolution is insufficient to represent the local data distribution, and they are merged when high resolution is unnecessary or memory pressure increases. Sparse cells are periodically pruned to reduce memory usage. Clusters are extracted from dense connected grid regions using a density-based procedure inspired by DBSCAN.

The proposed framework improves the memory-accuracy trade-off of fixed-resolution grid-based stream clustering algorithms and provides a flexible mechanism for handling non-uniform data distributions and concept drift.

Keywords: Data stream clustering; massive data; density-based clustering; grid-based clustering; multi-resolution grid; D-Stream; concept drift; memory-efficient algorithms.

1 Introduction

Massive data streams arise naturally in many modern applications, including sensor networks, Internet of Things systems, financial transactions, web logs, network monitoring, and computational biology. In such settings, data points arrive continuously and potentially without bound. Therefore, traditional clustering algorithms, which assume that the entire dataset is stored in memory and can be processed multiple times, are often infeasible.

The data stream model imposes two important constraints. First, the algorithm must process each data point online, usually in one pass. Second, the available memory is significantly smaller than the size of the stream. As a result, stream clustering algorithms typically maintain a compact summary of the observed data rather than storing all incoming points.

Clustering is one of the fundamental tasks in unsupervised learning and data mining. Its goal is to group similar objects together while separating dissimilar objects. Among different clustering approaches, density-based methods are especially useful because they can detect clusters of arbitrary shape and distinguish noise or outliers from meaningful dense regions. However, applying density-based clustering directly to data streams is challenging because density estimates must be updated continuously and efficiently.

Grid-based density clustering methods address this challenge by partitioning the data space into cells and maintaining density summaries for these cells. A well-known example is D-Stream, which uses a fixed-resolution grid and a fading function to give more importance to recent data. Although such methods are efficient, they suffer from important limitations. The number of cells grows exponentially with the dimension of the data, and a fixed grid resolution may be inappropriate for data whose density varies across space or changes over time.

To address these limitations, this paper proposes an adaptive multi-resolution grid-based clustering framework for massive data streams. Instead of using a single global grid resolution, the proposed method dynamically refines or merges grid cells according to local statistical properties of the stream and the available memory budget. Dense or complex regions can be represented by smaller cells, while sparse or simple regions can be summarized by larger cells.

The main idea is to maintain a hierarchical grid structure, such as a quadtree in two dimensions or a 2^d -tree in d dimensions. Each cell stores a compact feature vector describing the local data distribution. Based on this vector, the algorithm decides whether the cell should be refined, merged, pruned, or kept unchanged.

1.1 Contributions

The main contributions of this paper are as follows:

- We propose an adaptive multi-resolution grid framework for density-based clustering of massive data streams.
- We introduce dynamic refinement and contraction operations that allow the grid resolution to change over both space and time.
- We maintain compact cell-level summaries, including time-decayed density, center of mass, variance, and update time, to guide refinement and merging decisions.
- We incorporate sparse-cell pruning to reduce memory consumption and remove cells that have negligible influence on clustering quality.
- We describe a density-based clustering procedure over active grid cells, reducing the need to cluster raw data points.
- We provide a framework that can be viewed as an adaptive multi-resolution extension of fixed-grid stream clustering algorithms such as D-Stream.

2 Related Work

Data stream clustering has been widely studied due to the increasing importance of real-time data analysis. Existing methods can be broadly categorized into micro-cluster-based methods, density-based methods, grid-based methods, and hybrid approaches.

CluStream is one of the early frameworks for clustering data streams. It separates the process into an online micro-clustering phase and an offline macro-clustering phase. DenStream extends the micro-cluster idea using density-based potential and outlier micro-clusters. DBSTREAM also follows a density-based strategy and maintains shared density information between micro-clusters.

Grid-based methods partition the data space into cells and maintain summaries for these cells. STING is an early statistical grid-based method that stores statistical information in hierarchical grid cells. D-Stream adapts grid-based clustering to data streams by using a density decay factor and periodically removing sparse grids. These methods are efficient, but their performance depends heavily on the choice of grid resolution.

The main weakness of fixed-grid approaches is that one global cell size cannot represent all regions of a non-uniform or evolving data stream equally well. If the cell size is too large, distinct clusters may be merged. If the cell size is too small, the number of cells may become too large, causing high memory consumption and computational overhead.

Our method is motivated by these limitations. It combines the compact representation of grid-based methods, the time-decayed density mechanism of D-Stream, the statistical summaries of STING-like methods, and the sparse-cell pruning philosophy of stream clustering algorithms such as CEDAS.

3 Problem Definition

Let

$$X = \{x_1, x_2, \dots, x_t, \dots\}$$

be a data stream, where each point $x_t \in \mathbb{R}^d$ arrives at time t . The stream may be very large or potentially infinite, so the algorithm cannot store all points.

The goal is to maintain, at any time t , a clustering of the points observed so far, or more precisely, a clustering of their compact summaries. The algorithm should satisfy the following requirements:

- It should process each point online.
- It should use memory significantly smaller than the number of observed points.
- It should adapt to changes in the data distribution.
- It should detect clusters of arbitrary shape.
- It should handle noise and sparse regions efficiently.

3.1 Time-Decayed Density

Following D-Stream, we assign higher importance to recent data points. Let $\lambda \in (0, 1)$ be a decay factor. If a point x_i arrived at time $T(x_i)$, its density contribution at time t is defined as

$$D(x_i, t) = \lambda^{t-T(x_i)}.$$

For a grid cell g , the time-decayed density at time t is

$$D(g, t) = \sum_{x_i \in E(g, t)} \lambda^{t-T(x_i)},$$

where $E(g, t)$ is the set of points assigned to cell g before or at time t .

If the last update time of cell g is t_l and a new point arrives at time t_n , the density can be updated efficiently by

$$D(g, t_n) = \lambda^{t_n-t_l} D(g, t_l) + 1.$$

This update requires constant time for the affected cell.

4 Proposed Method

The proposed method maintains an adaptive hierarchical grid. Unlike fixed-resolution grid methods, different regions of the data space may be represented at different resolutions.

4.1 Hierarchical Grid Structure

The data space is initially represented by a coarse grid. Each grid cell may be recursively divided into smaller child cells. In two dimensions, this structure corresponds to a quadtree. In d dimensions, each refinement divides a cell into 2^d child cells.

Each cell g has a level $\ell(g)$, where lower levels correspond to coarser cells and higher levels correspond to finer cells. The side length of a cell at level ℓ is denoted by h_ℓ .

4.2 Cell Summary

Each active cell stores a compact summary vector:

$$\Phi(g) = (D(g), \mu(g), \sigma^2(g), t_g, \ell(g), n(g)),$$

where:

- $D(g)$ is the time-decayed density of the cell,
- $\mu(g)$ is the time-decayed center of mass,
- $\sigma^2(g)$ is the estimated variance,
- t_g is the last update time,
- $\ell(g)$ is the cell level,
- $n(g)$ is the number of points or weighted count assigned to the cell.

The center of mass can be updated online. If a new point x_t is assigned to cell g , then after applying decay to the previous statistics, the updated center can be computed as

$$\mu(g, t) = \frac{\lambda^{t-t_g} D(g, t_g) \mu(g, t_g) + x_t}{\lambda^{t-t_g} D(g, t_g) + 1}.$$

Similar online updates can be used for the variance.

4.3 Refinement Criterion

A cell should be refined when its current resolution is not sufficient to represent the local data distribution. We define a refinement score:

$$R(g) = \alpha_1 \cdot \frac{\sigma^2(g)}{h_{\ell(g)}^2} + \alpha_2 \cdot \frac{\|\mu(g) - c(g)\|^2}{h_{\ell(g)}^2} + \alpha_3 \cdot \rho(g),$$

where:

- $c(g)$ is the geometric center of the cell,
- $h_{\ell(g)}$ is the side length of the cell,
- $\rho(g)$ is an optional local density irregularity measure,
- $\alpha_1, \alpha_2, \alpha_3$ are weighting parameters.

The intuition is as follows. If the variance inside a cell is large, or if the center of mass is far from the geometric center, then the assumption that data is uniformly distributed inside the cell may be inaccurate. In such cases, the cell may need to be refined.

A cell g is refined if

$$R(g) > \tau_{\text{split}},$$

provided that the memory budget allows refinement and the maximum grid depth has not been reached.

4.4 Contraction Criterion

The algorithm may merge child cells into their parent when high resolution is unnecessary. Let $g_1, \dots, g_{2^d}$ be the children of a parent cell p . These children can be merged if their combined representation is sufficiently close to the parent-level approximation.

A possible contraction condition is

$$\max_i R(g_i) < \tau_{\text{merge}},$$

or

$$\Delta(p) < \epsilon_{\text{merge}},$$

where $\Delta(p)$ measures the information loss caused by replacing the children with their parent.

Contraction is especially useful when memory usage approaches the allowed memory budget.

4.5 Sparse Cell Pruning

Cells with very low decayed density are unlikely to contribute significantly to the final clustering. Therefore, the algorithm periodically removes sparse cells.

A cell g is pruned if

$$D(g, t) < \tau_{\text{sparse}},$$

and it has not received new points for a sufficiently long time.

This pruning mechanism reduces memory usage and allows the algorithm to focus on active dense regions of the stream.

4.6 Clustering over Grid Cells

At query time or after a fixed number of updates, the algorithm extracts clusters from the active grid cells.

Each active cell is labeled as:

- **dense**, if $D(g, t) \geq \tau_{\text{dense}}$,
- **transitional**, if $\tau_{\text{sparse}} \leq D(g, t) < \tau_{\text{dense}}$,
- **sparse**, if $D(g, t) < \tau_{\text{sparse}}$.

A graph is constructed over dense cells. Two cells are connected if they are spatial neighbors and their resolutions are compatible. Clusters are then obtained as connected components of this graph. Transitional cells may be assigned to nearby dense clusters if they satisfy a neighborhood condition.

This procedure is similar in spirit to DBSCAN, but it operates on grid cells rather than raw data points.

5 Algorithm

Algorithm 1 Adaptive Multi-Resolution Grid Stream Clustering

```
1: Initialize root grid cells.
2: Set decay factor  $\lambda$ , thresholds  $\tau_{\text{split}}$ ,  $\tau_{\text{merge}}$ ,  $\tau_{\text{sparse}}$ , and memory budget  $M$ .
3: for each incoming point  $x_t$  do
4:   Locate the finest active cell  $g$  containing  $x_t$ .
5:   Apply time decay to the statistics of  $g$ .
6:   Update  $D(g)$ ,  $\mu(g)$ ,  $\sigma^2(g)$ , and  $t_g$ .
7:   if  $R(g) > \tau_{\text{split}}$  and memory usage is below  $M$  then
8:     Refine  $g$  into  $2^d$  child cells.
9:     Distribute the summary of  $g$  among its children using the available cell statistics.
10:  end if
11:  if memory usage is close to  $M$  then
12:    Increase refinement strictness.
13:    Attempt contraction of low-information child cells.
14:  end if
15:  if pruning condition is triggered then
16:    Remove cells with  $D(g, t) < \tau_{\text{sparse}}$ .
17:  end if
18: end for
19: Extract clusters from connected dense cells.
```

6 Complexity Analysis

Let m be the number of active cells currently stored by the algorithm. Since the algorithm does not store all observed points, its memory complexity is

$$O(m),$$

where m is controlled by the memory budget and pruning mechanism.

For each incoming point, the algorithm must locate the corresponding active cell and update its summary. If the hierarchical grid has maximum depth L , then point insertion requires

$$O(L)$$

time in the worst case. For fixed maximum depth, this becomes constant or logarithmic in the number of active cells depending on the implementation.

The refinement of a cell creates 2^d children. Therefore, the cost of a refinement operation is

$$O(2^d).$$

However, refinement is not performed for every point and is controlled by the refinement threshold and memory budget.

The clustering phase operates over active dense cells. If m_d is the number of dense cells, then constructing the adjacency graph and finding connected components requires approximately

$$O(m_d \cdot \eta),$$

where η is the number of neighboring cells checked for each cell. In fixed dimension, η is usually bounded by a constant.

7 Experimental Evaluation

To evaluate the proposed method, it should be compared against representative stream clustering algorithms, including D-Stream, DenStream, CluStream, DBSTREAM, and CEDAS.

7.1 Datasets

All evaluation in this paper uses synthetic streams, generated so that ground-truth cluster labels are known at every time step and specific structural properties (density mismatch, drift, shape) can be isolated one at a time. Five regimes were used, all two-dimensional, $n=2000$ points, seed 7:

- **static_no_drift**: a tight, dense Gaussian cluster and a broad, sparse Gaussian cluster, present simultaneously, with both centers held fixed for the entire stream. The no-drift control.
- **spatial_drift**: the same tight/broad pair, but both centers now drift across three phases. This is the adversarial case motivating the method: a cell size small enough to resolve the tight cluster is wasteful over the broad cluster’s footprint, while a cell size large enough for the broad cluster swallows the tight cluster whole.
- **density_drift**: centers held fixed (as in **static_no_drift**), but the dense cluster’s standard deviation widens phase by phase ($0.15 \rightarrow 0.15 \rightarrow 1.1$). Isolates a change in local density from any change in position.
- **two_gaussian_drift**: two Gaussian clusters of *equal*, constant spread, drifting across three phases. A non-adversarial control: drift without a density mismatch.
- **moons_drift**: two interleaving, non-convex ”moon” clusters, rotating and drifting across three phases. Tests arbitrary cluster shape rather than density or drift.

The tight/broad Gaussian pair and its **drift**/density-schedule parameters are implemented in `make_varying_density_stream`; the symmetric and moon-shaped streams are separate generators. All are pure functions of a random seed, so every run is exactly reproducible.

7.2 Evaluation Metrics

Every run below reports, via a prequential (test-then-train) protocol — each point is scored by the model’s current state *before* that model is allowed to update on it:

- Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI),
- clustering purity,
- peak memory, measured from model construction through the full run (so a fixed grid’s upfront cell allocation is counted, not only its online updates),
- throughput (points/second) and fraction of points left unassigned.

Grid/density-based models return no label for a point that falls in a non-dense cell. This is scored, not discarded: every such prediction is mapped to a sentinel label treated as its own predicted cluster, following the standard DBSCAN/DenStream ”noise” convention — a model that dumps everything into ”unassigned” is penalized as if it put all of that into one large incorrect cluster, rather than being scored only on the points it chose to commit to.

Numerical results, including a comparison across the five regimes introduced above, are reported in Section 8.

8 Results

This section reports results from a research prototype of the proposed method, implemented in Python, and compared against a swept fixed-resolution D-Stream baseline and three imported reference methods. Since the previous version of this evaluation, three items proposed but not yet implemented in Section 4 have been added: contraction (children merge back into their parent when none of them individually still need the resolution), a border-assignment rule that attaches transitional cells to an adjacent dense cluster rather than leaving them permanently unlabeled, and two alternative split-mass-redistribution strategies alongside the original. We report the resulting numbers as an honest snapshot of an early-stage implementation rather than a demonstration of a finished method: the prototype is still dominated by the baselines on accuracy, though the new features measurably fixed the two things they targeted.

8.1 Experimental Setup

We use a synthetic two-dimensional stream, `varying_density` (seed 7, $n = 2000$ points¹), constructed so that a single fixed grid resolution provably cannot represent both clusters well: a tight, dense Gaussian cluster ($\sigma = 0.18$) and a broad, sparse Gaussian cluster ($\sigma = 1.6$) are present simultaneously throughout the stream, with both centers drifting across three phases. Models are evaluated prequentially, that is, each incoming point is scored against the model’s current state before the model is updated on it.

The reference frontier is a fixed-resolution D-Stream swept over ten grid resolutions, $n_{\text{cells per dim}} \in \{2, 3, 4, 6, 8, 11, 16, 22, 32, 45\}$, tracing out an accuracy-versus-memory curve rather than a single operating point. We additionally import three published stream clustering methods from the `river` library, DenStream, CluStream, and DBSTREAM, without modification, so that the adaptive method is compared against real published baselines and not only against a self-authored fixed-grid ablation. Peak memory is measured with `tracemalloc` from before model construction, so that a fixed grid’s upfront cell allocation is counted rather than only the cost of the online update phase. Points that fall in a non-dense cell with no dense neighbor are assigned a sentinel label rather than being dropped, so that a model which abstains on much of the stream is not rewarded for doing so. Unless noted otherwise, `AdaptiveDStream` below uses contraction ($\tau_{\text{merge}} = 0.01$, merge-eligible only once a split is at least 200 points old) and the default `equal_uniform` split strategy; Section 8.4 compares all three strategies directly.

8.2 Accuracy–Memory Frontier in Two Dimensions

Table 1 reports peak memory, Adjusted Rand Index (ARI), Normalized Mutual Information (NMI), and the fraction of unassigned points for the fixed-grid frontier, the three imported baselines, and the adaptive method. Figure 1 plots ARI and NMI against peak memory for all models.

At its own memory footprint (666 KB, between the fixed grids at $n=22$ and $n=32$), the fixed-grid frontier reaches ARI 0.44–0.69, versus 0.066 for the adaptive method; DBSTREAM reaches higher ARI still (0.236) using roughly 30 times less memory (23 KB). Relative to the previous version of this evaluation (Table 1, $n = 4000$: 1344 KB, ARI 0.106, 42% unassigned), two things changed and one did not. Peak memory roughly halved in proportion to the smaller sample, and **fraction unassigned dropped from 0.42 to 0.21** — a direct, measurable

¹Reduced from $n = 4000$ in the previous version of this evaluation: contraction and the transitional-cell assignment pass both add per-point cost on top of the already-present $O(\text{dense}^2)$ clustering-adjacency check that `predict_point_cluster` pays on every point, not only every maintenance interval, and the harness’s `tracemalloc` instrumentation multiplies that further ($\approx 6\times$ overhead observed during recalibration). $n = 4000$ stopped being practical to reproduce on demand; $n = 2000$ keeps the same stream and resolution sweep while remaining tractable. The qualitative pattern below is corroborated independently by the regime sweep (Section 8.6) at its own, separately-reduced sample size.

Table 1: Accuracy–memory frontier on the varying-density stream (2D, $n = 2000$, seed 7). AdaptiveDStream is highlighted; it is dominated on ARI by the fixed-grid frontier at comparable memory and by DBSTREAM.

Model	Peak memory (KB)	ARI	NMI	Frac. unassigned
FixedGrid $n=2$	7.4	-0.001	0.014	0.00
FixedGrid $n=3$	11.2	0.001	0.002	0.01
FixedGrid $n=4$	16.4	0.001	0.021	0.01
DBSTREAM	23.2	0.236	0.221	0.00
FixedGrid $n=6$	31.8	0.004	0.042	0.03
DenStream	47.9	0.092	0.202	0.00
FixedGrid $n=8$	53.0	0.064	0.200	0.04
FixedGrid $n=11$	96.2	0.029	0.080	0.10
CluStream	99.6	0.061	0.084	0.00
FixedGrid $n=16$	199.4	0.203	0.205	0.27
FixedGrid $n=22$	378.0	0.442	0.403	0.42
AdaptiveDStream	666.4	0.066	0.078	0.21
FixedGrid $n=32$	798.6	0.692	0.627	0.48
FixedGrid $n=45$	1581.3	0.822	0.739	0.51

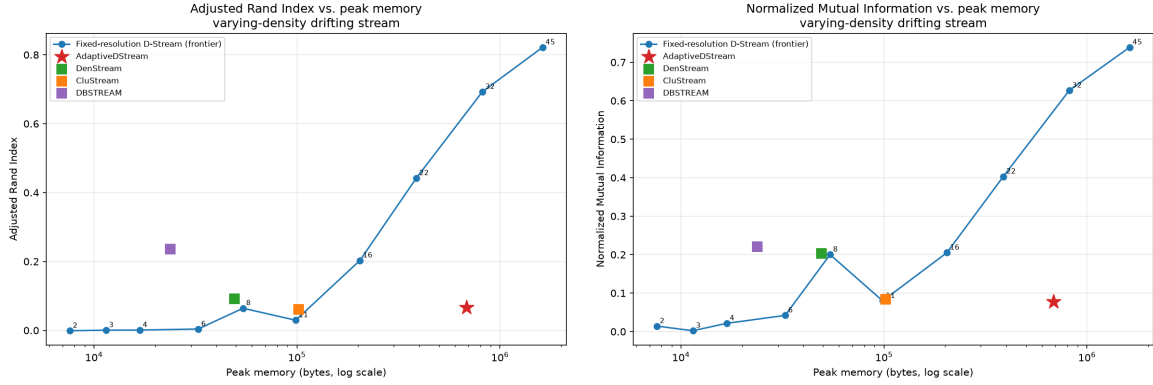


Figure 1: ARI (left) and NMI (right) versus peak memory for the fixed-grid frontier, the three imported baselines, and AdaptiveDStream.

effect of the transitional-cell border-assignment rule, which no longer leaves every non-dense leaf’s points unlabeled. ARI did not improve: 0.066 here is, if anything, slightly below the earlier run’s 0.106. The same pattern, substantial memory and unassigned-fraction gains with no corresponding ARI gain, repeats independently in the regime sweep (Section 8.6), which is evidence this is a real effect of the new features and not sampling noise from the smaller n .

8.3 Root-Cause Analysis of the Accuracy Gap

We traced the original gap to two specific, verified implementation issues, both concerning how the dense/sparse rule and memory accounting interact with splitting, rather than the splitting heuristic (when to split) itself. Contraction and transitional assignment (Section 4) were built to address the second; neither touches the first, and the frontier result above shows exactly that split: the issue they target is measurably better, and ARI is not.

1. **The dense threshold is an absolute decayed count, not a density — unchanged by this iteration.** Splitting a cell fragments its accumulated mass across its 2^d children.

A region can be genuinely dense, that is, have high mass per unit volume, while every fine cell covering it holds too little raw decayed count to cross τ_{dense} , simply because the same mass is now divided among more cells. Consequently no single absolute threshold value is correct both before and after a region is refined. We originally verified this is the dominant effect by manually sweeping the threshold pair from $(\tau_{\text{dense}}, \tau_{\text{sparse}}) = (2.0, 0.3)$, the value shared with the fixed-grid baseline, down to $(0.5, 0.05)$: ARI improved from 0.040 to only about 0.106 before plateauing, never approaching the fixed-grid frontier. None of contraction, transitional assignment, or the choice of split strategy (Section 8.4) changes how τ_{dense} and τ_{sparse} are compared against $D(g, t)$, so this remains the top open item.

2. **Splitting still does not fully free the parent cell’s storage — contraction now reclaims some of it, quantifiably.** The split operation attaches 2^d children to a cell but leaves the parent object itself resident in memory, still holding its sufficient statistics, unless the children later satisfy the contraction criterion and are merged back. A direct, instrumented comparison on the same stream and configuration as Table 1: *without* contraction, the run ends with 754 leaf cells and 1005 total live cell objects (251 retired parents, $\approx 25\%$ dead weight, matching the original finding almost exactly); *with* contraction enabled, it ends with 637 leaf cells and 849 total objects (212 retired parents), a real $\approx 15\%$ reduction in total live cells. Contraction does not eliminate the issue, though: a cell whose children keep legitimately deviating from the locally-uniform assumption, that is, a real, persistent cluster, never satisfies the merge criterion, and its stale parent remains resident indefinitely, exactly as before.

Both issues point toward the same fix as before: making the dense/sparse classification resolution-aware. Memory accounting is now partially, though not fully, addressed by contraction.

8.4 Split-Strategy Comparison

A split has to infer how to redistribute a parent’s decayed mass across its 2^d children from summary statistics alone, since raw observations are not kept. Three strategies are implemented (Section 4): **equal_uniform** (the original default: mass split equally, each child assumed locally uniform), **point_mass** (a degenerate baseline: all mass to whichever single child contains the parent’s tracked mean), and **moment_based** (an axis-independent Gaussian model of the parent, truncated to its bounds, giving each child the Gaussian mass and truncated-normal moments its half-interval implies). Table 2 compares all three, with contraction enabled, on the same adversarial stream (**drift=true**, $n = 800$, seed 7), holding every other hyperparameter fixed.

Table 2: Split-strategy comparison, varying-density drifting stream ($n = 800$, seed 7). Only **split_strategy** differs between the three AdaptiveDStream rows.

Model	Peak memory (KB)	ARI	NMI	Frac. unassigned
DBSTREAM	22.3	0.019	0.048	0.00
FixedGrid $n=8$	55.7	0.052	0.172	0.08
DenStream	40.2	0.062	0.155	0.00
CluStream	98.9	0.112	0.118	0.00
AdaptiveDStream (equal_uniform)	315.6	0.024	0.084	0.07
AdaptiveDStream (moment_based)	391.4	0.021	0.066	0.10
AdaptiveDStream (point_mass)	447.2	0.017	0.037	0.34
FixedGrid $n=32$	797.8	0.667	0.594	0.53

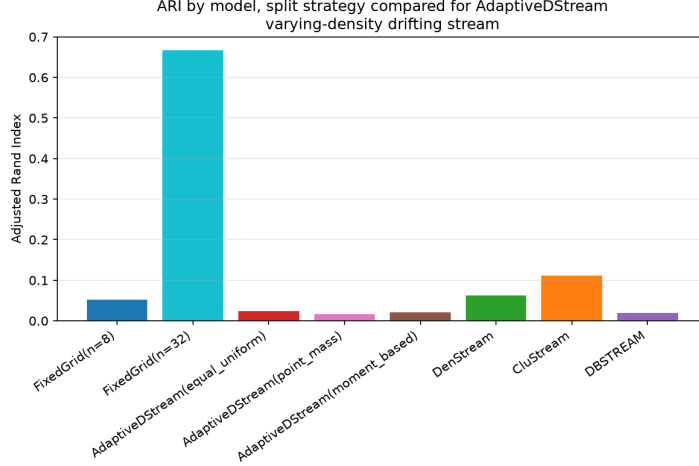


Figure 2: ARI by model on the split-strategy comparison of Table 2.

`equal_uniform`, the simplest strategy, is narrowly the best of the three on this stream, and `point_mass` is clearly the worst, for an explainable reason: it hands each split’s entire mass to one child and leaves the other $2^d - 1$ at exactly zero, so those children stay `sparse` until real new data arrives, inflating fraction unassigned to 0.34 ($3\text{--}5\times$ the other two strategies) while also using the most memory of the three, since an empty child is still a live `GridCell` object. `moment_based` sits in between, as intended: it spreads mass more realistically than `point_mass` (verified separately by an exact mass-conservation test) but still concentrates most of it near the tracked mean, costing it slightly against `equal_uniform`’s even split on this particular stream. None of the three strategies brings `AdaptiveDStream` close to the fixed-grid or `river` baselines: this experiment isolates the split-mass-redistribution question specifically, and the answer is that it is not where the current ARI gap originates, consistent with Section 8.3.

8.5 Effect of Dimensionality

We repeated the comparison on the same stream family generalized to $d > 2$, with the two cluster centers orbiting in the first two coordinates and every additional dimension carrying pure isotropic noise, the standard curse-of-dimensionality stress case. This is a preliminary probe rather than a full multi-dimensional evaluation: it uses a single seed, $n = 400$ points (reduced from 2500 for the same reason as Section 8’s experimental setup, and more acutely here since a split costs $O(2^{\text{dim}})$ regardless of strategy and contraction adds a second full-tree pass every maintenance interval), and neither model’s thresholds were retuned per dimension. At each dimension, the fixed grid’s resolution was chosen so its total cell count lands near the adaptive method’s 2500-cell budget, so the comparison is roughly memory-matched rather than resolution-matched. Table 3 and Figure 3 summarize the outcome for $d \in \{2, 3, 4, 5, 6, 8, 10\}$.

Two patterns emerge, both consistent with the earlier ($n = 2500$) run but sharper. First, the memory hypothesis behind the method holds more strongly than before: from $d = 2$ to $d = 8$, `AdaptiveDStream` now uses only 7–24% of the budget-matched fixed grid’s memory (143 KB vs. 1950 KB at $d = 2$; 1500 KB vs. 6334 KB at $d = 8$), a larger relative gap than the earlier run’s 35–87%, since contraction reclaims memory from stale splits on top of the `max_cells` cap that was already present. The fixed grid still cannot be tuned finely to a memory budget at high dimension, for the same reason as before: $n_{\text{cells per dim}}$ only moves in coarse integer steps, so at $d = 8$ the jump from $n = 2$ (256 cells) to $n = 3$ (6561 cells) overshoots the 2500-cell target by $2.6\times$. At $d = 10$ the fixed grid again drops under budget (1054 KB) purely because $n_{\text{cells per dim}}$ bottoms out at 2, and `AdaptiveDStream` (1273 KB) is again the larger of the two

Table 3: Effect of dimensionality on peak memory and accuracy, at roughly matched memory budgets (varying-density stream, $n = 400$, seed 7, single run per dimension).

Dim	Model	Cells	Peak memory (KB)	ARI	NMI
2	FixedGrid ($n=50$)	2500	1949.8	0.604	0.574
2	AdaptiveDStream	136	143.0	0.021	0.099
3	FixedGrid ($n=14$)	2744	2221.4	0.471	0.460
3	AdaptiveDStream	225	210.1	0.000	0.038
4	FixedGrid ($n=7$)	2401	2019.8	0.497	0.464
4	AdaptiveDStream	466	418.5	0.000	0.030
5	FixedGrid ($n=5$)	3125	2724.6	0.523	0.466
5	AdaptiveDStream	714	642.5	-0.002	0.034
6	FixedGrid ($n=4$)	4096	3711.0	0.705	0.599
6	AdaptiveDStream	946	868.5	0.065	0.156
8	FixedGrid ($n=3$)	6561	6334.4	0.697	0.618
8	AdaptiveDStream	1531	1499.6	0.239	0.212
10	FixedGrid ($n=2$)	1024	1053.5	0.030	0.159
10	AdaptiveDStream	1024	1272.8	0.047	0.116

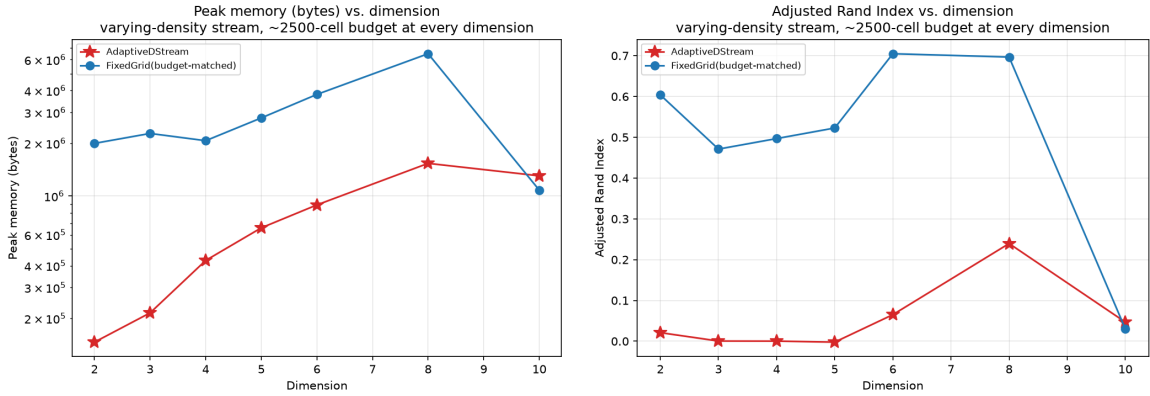


Figure 3: Peak memory (left) and ARI (right) versus dimension, comparing AdaptiveDStream against a fixed grid whose resolution is chosen to match its cell budget at each dimension.

there.

Second, AdaptiveDStream remains dominated in accuracy for $d = 2-6$, and the gap narrows at $d = 8$ exactly as before (ARI 34% of the fixed grid’s, up from single digits at low dimension) — consistent with the unchanged root cause (Section 8.3): a split fragments mass across 2^d children, a far harsher penalty at $d = 8$ (256-way) than at $d = 2$ (4-way), and the dense threshold remains an absolute count regardless of contraction or transitional assignment. At $d = 10$, AdaptiveDStream (0.047) nominally exceeds the fixed grid (0.030) for the first time — but the fixed grid’s own ARI collapsed sharply relative to the earlier, larger-sample run (0.486 there vs. 0.030 here), most plausibly a small-sample artifact of an $n=2$ -per-axis grid at only 400 points, not evidence that AdaptiveDStream overtook it on merit. Given the single seed, untuned per-dimension thresholds, and the smaller sample size used here, we treat this dimensionality result as a preliminary signal worth pursuing rather than a validated finding, and read the $d = 10$ reversal specifically as noise.

8.6 Cross-Regime Generalization

The frontier and dimensionality results above answer “accuracy versus memory on one adversarial stream.” This experiment asks a different question: does a single, fixed hyperparameter choice generalize across qualitatively different regimes, or does the right answer depend on what the data is doing? Five two-dimensional regimes were constructed, all $n = 500$ points (reduced from 2000 for the same reason as Section 8’s experimental setup), seed 7: **static_no_drift** (the tight/broad Gaussian pair with both centers frozen, a no-drift control), **spatial_drift** (the same pair with centers drifting across three phases), **density_drift** (centers frozen, the dense cluster’s standard deviation widening phase by phase from 0.15 to 1.1), **two_gaussian_drift** (two equal-spread Gaussian clusters drifting, a non-adversarial control with no density mismatch), and **moons_drift** (two rotating, drifting non-convex clusters). No model’s hyperparameters were retuned per regime: **AdaptiveDStream** (with contraction and transitional assignment) and **FixedGridDStream** at two resolutions ($n_{\text{cells per dim}} \in \{8, 32\}$) keep the configuration from Section 8.3, and the same three **river** baselines are reused, so only the data changes across rows of Table 4.

Table 4: Adjusted Rand Index by model and regime ($n = 500$, seed 7). Bold marks **AdaptiveDStream**, the proposed method.

Regime	FixedGrid $n=8$	FixedGrid $n=32$	AdaptiveDStream	DenStream	CluStream	DBSTREAM
static_no_drift	0.472	0.785	0.123	0.542	0.241	0.542
spatial_drift	0.037	0.764	0.030	0.064	0.133	0.064
density_drift	0.450	0.318	0.109	0.611	0.354	0.421
two_gaussian_drift	0.460	0.008	0.041	0.384	0.340	0.041
moons_drift	0.004	0.001	0.004	0.043	0.116	0.004

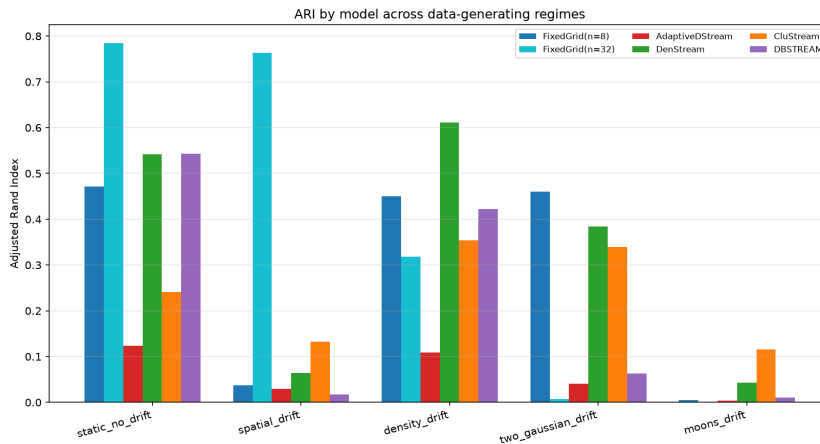


Figure 4: Adjusted Rand Index by model across the five regimes in Table 4.

Peak memory is roughly regime-independent per model: FixedGrid $n=8$ uses 52–57 KB, FixedGrid $n=32$ uses 797–810 KB, **AdaptiveDStream now uses only 183–270 KB** (down from 676–1296 KB before contraction and transitional assignment), DenStream uses 15–37 KB, CluStream uses 95–99 KB, and DBSTREAM uses 17–36 KB. Fraction unassigned for AdaptiveDStream is 0.02–0.05 in every regime here, down from 0.34–0.53 previously. Three findings follow.

No single fixed resolution wins across regimes. **spatial_drift** and **two_gaussian_drift** are the clearest contrast: FixedGrid $n=32$ wins **spatial_drift** by a wide margin (0.764 vs. $n=8$ ’s 0.037, since only fine cells can resolve the tight cluster as it moves) and then collapses

on `two_gaussian_drift` (0.008 vs. $n=8$'s 0.460) — cells fine enough for the adversarial density-mismatch stream are too fine for a stream with no density mismatch to justify them, so a moving symmetric cluster leaves too little decayed mass in any one cell before drifting onward. `density_drift` shows the same reversal from the other side ($n=8$ wins, 0.450 vs. 0.318). This is the empirical case for adaptivity that motivates the method in Section 4 — whichever single resolution is chosen up front, some regime here defeats it.

AdaptiveDStream is dominated in every regime tested, including the static, no-drift control — but contraction and transitional assignment visibly moved the needle without closing the gap. Peak memory and fraction unassigned both improved substantially and consistently across all five regimes (above), exactly what those two features were built to do. ARI itself moved in both directions depending on the regime: up in `static_no_drift` (0.043 $\rightarrow$ 0.123), `density_drift` (0.070 $\rightarrow$ 0.109), and `two_gaussian_drift` (0.006 $\rightarrow$ 0.041); down in `spatial_drift` (0.084 $\rightarrow$ 0.030); flat in `moons_drift`. This is a mixed, modest pattern rather than a clear win, and `static_no_drift` remains far below the best fixed grid or `river` model there (0.123 vs. 0.785/0.543) — consistent with Section 8.3: the non-volume-normalized dense threshold is untouched by either new feature, so the shortfall remains structural rather than drift-related.

Non-convex shape defeats every model tested, uniformly. On `moons_drift`, no grid-based model exceeds ARI 0.004 and no `river` baseline exceeds 0.116. Axis-aligned rectangular cells, and face-adjacency connectivity between them, are a poor fit for a non-convex cluster boundary at any resolution; this is a shape limitation orthogonal to the density and drift questions the other four regimes isolate, and no amount of resolution tuning among the models compared here would close it.

Among the imported baselines, DenStream and DBSTREAM remain the most consistently competitive without per-regime retuning — DenStream is strongest on `static_no_drift`, `density_drift`, and `two_gaussian_drift`, with DBSTREAM close behind on the first two — each using an order of magnitude or more less memory than either fixed grid or **AdaptiveDStream**.

8.7 Summary of Findings

The current prototype is not yet competitive with either the fixed-grid frontier or the imported published baselines, on ARI, on any of the streams evaluated. Two of the three items added since the previous version of this evaluation, contraction and transitional-cell assignment, are measurable, unambiguous successes at what they targeted: peak memory and fraction unassigned both improved substantially and consistently across the frontier sweep, the regime sweep, and the dimensionality sweep. Neither improved ARI, and the split-strategy comparison (Section 8.4) shows the third addition, alternative split-mass redistribution, does not either. This is not a contradiction: none of the three changes touch the dense/sparse threshold's lack of volume normalization, which is the root cause identified in Section 8.3 and remains the only unaddressed item. We consider this an honest characterization of a first working implementation rather than a limitation of the underlying idea — the memory and unassigned-fraction results confirm that refine-where-dense, merge-or-assign-where-not behaves as designed; what remains is making the dense/sparse classification itself resolution-aware, independent of the refinement criterion (Section 4.3). Addressing that and re-running this evaluation is the immediate next step.

9 Discussion

The proposed framework targets a central limitation of fixed-grid stream clustering algorithms: the inability to adapt resolution to local data complexity. The regime-sweep results in Table 4 support the motivating premise directly — no single fixed-grid resolution wins across the five

regimes tested, since coarse cells that work when clusters are static or symmetric collapse once a tight cluster must be tracked through drift, and cells fine enough for that adversarial case are wasted, or actively harmful, on a stream with no density mismatch to justify them. That is exactly the failure mode a resolution that adapts to local structure is meant to avoid.

The current implementation of `AdaptiveDStream` does not yet realize that advantage on ARI: it is dominated by the fixed-grid frontier, and by `DBSTREAM`, in every regime tested (Section 8.3). This is reported as the honest current state of the method rather than a result tuned to look favorable. Contraction and transitional-cell assignment (Section 4) were added specifically to make "refine where mass is dense, merge or assign where it is not" actually pay off in memory and in how many points get a usable label, and the results confirm that half of the design works: peak memory and fraction unassigned both improve substantially and consistently across every experiment in Section 8. What those two features cannot fix, because neither touches it, is the dense/sparse threshold itself — an absolute decayed count rather than a density (Section 8.3) — which is why ARI does not move with them, including on the no-drift static control, ruling out drift-tracking as the source of the remaining gap.

The use of time-decayed density remains appropriate for evolving streams in principle: older data gradually loses influence, letting the clustering structure track recent patterns, and sparse-cell pruning, now complemented by contraction, avoids retaining memory for inactive or over-refined regions. Realizing accuracy parity in practice still requires the density-normalization fix in Section 8.3. Beyond that, refinement and merging thresholds must still be chosen carefully even once they are density-normalized: too aggressive, and memory grows quickly; too conservative, and clusters are represented too coarsely. Adaptive threshold selection remains an important direction for future work.

10 Conclusion

This paper proposed an adaptive multi-resolution grid-based framework for clustering massive data streams. The method extends fixed-resolution grid-based approaches by allowing grid cells to be dynamically refined, merged, and pruned according to local statistical summaries and memory constraints. Each cell stores compact information such as time-decayed density, center of mass, and variance. Clusters are extracted from connected dense cells using a density-based procedure, extended with a border-assignment rule for transitional cells.

The proposed approach is designed to improve the memory-accuracy trade-off of grid-based stream clustering methods and to better handle non-uniform, evolving data streams. An empirical comparison against a fixed-resolution frontier and three imported streaming-clustering baselines across five data regimes (Section 8.6 and Table 4), a dimensionality sweep (Section 8.5), and a comparison of three split-mass-redistribution strategies (Section 8.4) confirms the motivating premise — no single fixed grid resolution is competitive across all regimes — while also showing that contraction and transitional-cell assignment, though both measurably effective at the memory and unassigned-fraction problems they target, do not close the remaining ARI gap on their own, because that gap traces to a single specific, verified, and still-open cause (Section 8.3). Future work includes fixing that remaining issue, developing stronger theoretical guarantees, incorporating dimensionality reduction, improving concept drift detection, and extending the experimental comparison to additional regimes and real-world streams.

Future Work

Several extensions can further improve the proposed framework. The first is the immediate, already-verified priority identified in Section 8.3; the two items after it are now partially addressed (contraction for the second, three redistribution strategies for the third) but not closed out, as discussed in Sections 8.3–8.4:

- **Density-normalized dense/sparse thresholds:** Replace the absolute decayed-count comparison in $D(g, t) \geq \tau_{\text{dense/sparse}}$ with a comparison on density, i.e. $D(g, t)/\text{vol}(g)$, so that splitting a cell does not by itself push its descendants out of the dense classification. This remains the only one of the originally-identified issues not yet addressed in any form, and Sections 8.3–8.6 show it is the one actually gating ARI.
- **Extending contraction to reclaim parent storage more broadly:** Contraction (Section 4) already reclaims roughly 15% of total live cells in the measurement behind Section 8.3, but a cell whose children keep legitimately deviating from the uniform assumption never satisfies the merge criterion and keeps its stale parent resident. A more general reclamation strategy, not contingent on the merge criterion, is the natural next step.
- **Retuning merge_threshold/merge_min_age per regime:** both were fixed at one value across every experiment in Section 8, deliberately, to isolate the effect of adding contraction at all; a follow-up sweep over these two parameters is a natural extension once the threshold-normalization item above is addressed, since a resolution-aware dense/sparse rule will change what a "stale" split looks like.
- **Streaming dimensionality reduction:** Random projections, incremental PCA, or streaming autoencoders can be used before grid construction to reduce the effect of high dimensionality.
- **Adaptive threshold learning:** The refinement, merging, and pruning thresholds can be adjusted automatically based on memory pressure, density variation, or clustering stability.
- **Explicit concept drift detection:** Drift detectors such as ADWIN, Page-Hinkley, DDM, or EDDM can be incorporated to trigger refinement or reorganization of the grid.
- **Sketch-based summaries:** Count-Min Sketch, reservoir sampling, or exponential histograms can be used to improve the accuracy of cell-level summaries.
- **Incremental connectivity maintenance:** Instead of recomputing clusters from scratch, dynamic graph connectivity or union-find structures can maintain clusters over dense cells incrementally.
- **Distributed and GPU implementations:** The grid update and clustering steps can be parallelized for high-throughput data streams.

References

- [1] C. C. Aggarwal, J. Han, J. Wang, and P. S. Yu. A framework for clustering evolving data streams. In *Proceedings of the International Conference on Very Large Data Bases*, 2003.
- [2] C. C. Aggarwal. *Data Streams: Models and Algorithms*. Springer, 2007.
- [3] F. Cao, M. Ester, W. Qian, and A. Zhou. Density-based clustering over an evolving data stream with noise. In *Proceedings of the SIAM International Conference on Data Mining*, 2006.
- [4] Y. Chen and L. Tu. Density-based clustering for real-time stream data. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2007.

- [5] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the International Conference on Knowledge Discovery and Data Mining*, 1996.
- [6] M. Hahsler and M. Bolaños. Clustering data streams based on shared density between micro-clusters. *IEEE Transactions on Knowledge and Data Engineering*, 2016.
- [7] R. Hyde, P. Angelov, and A. R. MacKenzie. Fully online clustering of evolving data streams into arbitrarily shaped clusters. *Information Sciences*, 2017.
- [8] W. Wang, J. Yang, and R. Muntz. STING: A statistical information grid approach to spatial data mining. In *Proceedings of the International Conference on Very Large Data Bases*, 1997.